- Command-line arguments are divided into two types: positional arguments and flags. Flags are typically optional and are denoted by a leading dash or double-dash.
- Django automatically creates a `--help` argument for your command and displays the message found within your command class's `help` attribute.
- The `BaseCommand` class wraps Python's `argparse` library for you, creating a command-line argument parser and parsing any additional arguments to your command.
- You can register command-line positional arguments and flags with the `argparse` parser by overriding the command class's `.add_arguments()` method. The method gets called with a reference to the parser, which you use to register arguments through its `.add_argument()` method.
- The `.handle()` method gets passed `options`, which contains the positional arguments and flags passed on the command-line to your management command.
- If you raise a `CommandError` in your management command, Django suppresses its stack trace, printing a colorized message instead. This behavior can be overridden using the `--traceback` flag.
- The `call_command()` function allows you to invoke a management command programmatically. This is particularly helpful for writing unit tests for your command.
- You can capture the output from a management command by passing an `io.String` stream to the `stdout` and `stderr` parameters of the `call_command()` function.